

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №2
по курсу «Логика и основы алгоритмизации в
инженерных задачах»
на тему «Оценка выполнения работы
программы»

Выполнил:

студенты группы 24BBB2

Бауэр А.А.

Гусев Е.В.

Медведев Е.А.

Принял:

доцент Юрова О.В.

к.н. Деев М.В.

Пенза 2025

Название

Оценка времени выполнения программы.

Цель работы

Научиться оценивать время выполнения работы программы.

Лабораторное задание

1 задание:

- 1.1. Вычислить порядок сложности программы (О-символику).
- 1.2. Оценить время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки `time.h` для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000.
- 1.3. Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

2 задание:

- 2.1. Оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива.
- 2.2. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел.
- 2.3. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел.
- 2.4. Оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, – убывающую.
- 2.5. Оценить время работы стандартной функции `qsort`, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

Описание метода решения задачи

Программа умножения матриц использует классический алгоритм с тройным вложенным циклом сложности $O(n^3)$. Тестирует производительность на матрицах размеров от 100 на 100 до 4000 на 4000. Измеряет время выполнения, чтобы показать экспоненциальный рост времени с увеличением размера матриц.

Программа сравнения сортировок тестирует три алгоритма (Шелла, быструю и `qsort`) на четырёх типах данных (случайные, возрастающие, убывающие, смешанные) и пяти размерах массивов (от 1000 до 100000 элементов). Измеряет время работы каждого алгоритма в разных условиях, чтобы сравнить их эффективность и выявить оптимальные сценарии использования.

Анализ и выполнение заданий

- 1.1. Программа имеет три вложенных цикла для умножения матриц. Из этого можно сделать вывод, что сложность программы $O(n^3)$.
- 1.3. Эксперимент подтвердил кубическую сложность $O(n^3)$ алгоритма умножения матриц для размеров 100-2000. Для матриц 4000 на 4000 время выполнения (2444 сек) в 5 раз превысило теоретическую оценку (481 сек) из-за переполнения кэш-памяти и использования виртуальной памяти. Данный алгоритм неэффективен для больших матриц. (Рисунок 1)

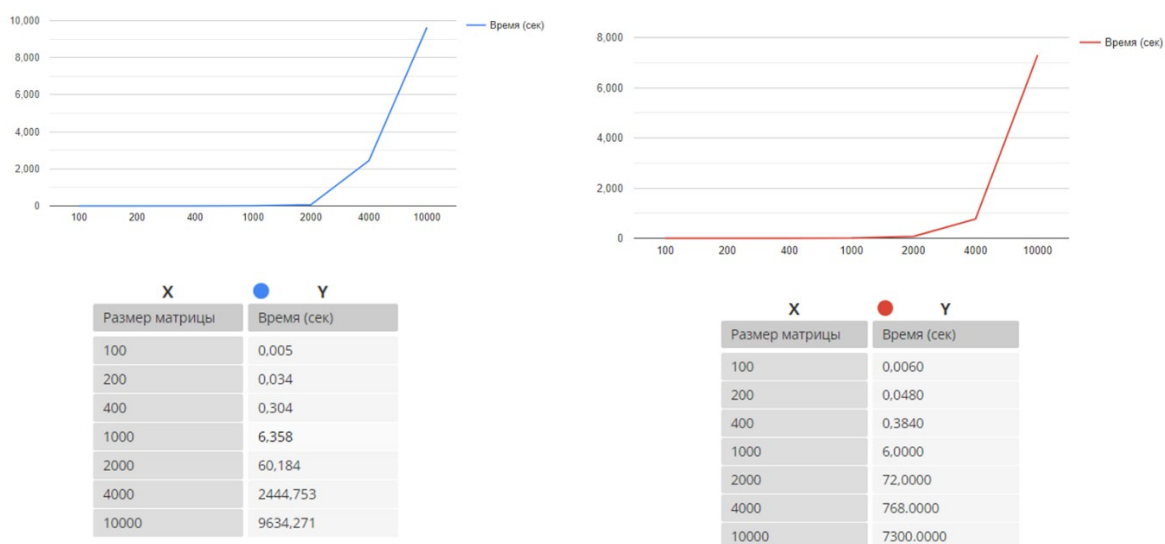


Рисунок 1 – График практического и теоретического времени, соответственно

- 2.1 Все алгоритмы эффективны на малых массивах (до 5000 элементов), но на больших объемах быстрая сортировка показывает наилучшие результаты (0,010 сек для 100000 элементов). Сортировка Шелла значительно медленнее (0,928 сек), qsort демонстрирует стабильные средние результаты (0,019 сек).
- 2.2 Все алгоритмы отлично справляются с отсортированными данными. Время выполнения минимальное (0,000-0,010 сек) даже для больших массивов, что свидетельствует об эффективной обработке уже упорядоченных последовательностей.
- 2.3 Сортировка Шелла значительно замедляется (1,855 сек для 100000 элементов). Быстрая сортировка неожиданно эффективна (0,004 сек), qsort показывает стабильно хорошие результаты (0,011 сек).
- 2.4 Быстрая сортировка обрабатывает массив достаточно долго (5,809 сек) из-за плохого выбора опорного элемента. Сортировка Шелла умеренно эффективна (0,927 сек). qsort демонстрирует наилучшие результаты (0,037 сек) благодаря защите от худших случаев.
- 2.5 Показала исключительную надежность на всех типах данных.

Оптимизирована для избежания деградации, содержит advanced алгоритмы выбора опорного элемента. Наименее подвержена влиянию паттернов данных.

Выводы по работе программы:

Сортировка Шелла - лучший выбор для малых и средних массивов (до 10000 элементов) и частично упорядоченных. Низкая производительность на больших объемах.

Быстрая сортировка - оптимальна для случайных данных среднего и большого размера (5000-100000 элементов).

Стандартная qsort - универсальное решение для всех объемов и типов данных. Наилучшая надежность и стабильность, особенно на больших массивах.

Листинг

Файл 2lab_12task.c (1 задание)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

//Структура для хранения матрицы
typedef struct {
    int** data;
    int size;
} Matrix;

//Выделение памяти для матрицы
Matrix* create_matrix(int n) {
    Matrix* matrix = (Matrix*)malloc(sizeof(Matrix));
    if (matrix == NULL) return NULL;

    matrix->size = n;
    matrix->data = (int**)malloc(n * sizeof(int*));
    if (matrix->data == NULL) {
        free(matrix);
        return NULL;
    }

    for (int i = 0; i < n; i++) {
        matrix->data[i] = (int*)malloc(n * sizeof(int));
        if (matrix->data[i] == NULL) {
            // Освобождаем уже выделенную память в случае ошибки
            for (int j = 0; j < i; j++) {
                free(matrix->data[j]);
            }
            free(matrix->data);
            free(matrix);
            return NULL;
        }
    }
}
```

```

    }
}
return matrix;
}

//Освобождение памяти матрицы
void free_matrix(Matrix* matrix) {
    if (matrix == NULL) return;

    for (int i = 0; i < matrix->size; i++) {
        free(matrix->data[i]);
    }
    free(matrix->data);
    free(matrix);
}

//Инициализация матрицы случайными числами
void init_matrix_random(Matrix* matrix) {
    if (matrix == NULL) return;

    for (int i = 0; i < matrix->size; i++) {
        for (int j = 0; j < matrix->size; j++) {
            matrix->data[i][j] = rand() % 100 + 1;
        }
    }
}

//Инициализация матрицы нулями
void init_matrix_zero(Matrix* matrix) {
    if (matrix == NULL) return;

    for (int i = 0; i < matrix->size; i++) {
        for (int j = 0; j < matrix->size; j++) {
            matrix->data[i][j] = 0;
        }
    }
}

//Умножение матриц: C = A на B
void multiply_matrices(const Matrix* a, const Matrix* b, Matrix* c)
{
    if (a == NULL || b == NULL || c == NULL || a->size != b->size ||
a->size != c->size) {
        return;
    }

    int n = a->size;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            c->data[i][j] = 0;
            for (int k = 0; k < n; k++) {
                c->data[i][j] += a->data[i][k] * b->data[k][j];
            }
        }
    }
}

```

```

//Основная функция тестирования
int main() {
    setlocale(LC_ALL, "");
    srand(time(NULL));
    int sizes[] = { 100, 200, 400, 1000, 2000, 4000, 10000 };
    int num_sizes = sizeof(sizes) / sizeof(sizes[0]);
    printf("Размер матрицы | Время (сек)\n");
    printf("-----\n");
    for (int i = 0; i < num_sizes; i++) {
        int n = sizes[i];

        //Создание матриц
        Matrix* a = create_matrix(n);
        Matrix* b = create_matrix(n);
        Matrix* c = create_matrix(n);
        if (a == NULL || b == NULL || c == NULL) {
            printf("Ошибка выделения памяти для размера %d\n", n);
            free_matrix(a);
            free_matrix(b);
            free_matrix(c);
            continue;
        }

        //Инициализация матриц
        init_matrix_random(a);
        init_matrix_random(b);
        init_matrix_zero(c);

        //Замер времени только умножения
        clock_t start = clock();
        multiply_matrices(a, b, c);
        clock_t end = clock();
        double time_taken = ((double)(end - start)) /
CLOCKS_PER_SEC;
        printf("%13d | %10.4f\n", n, time_taken);

        //Освобождение памяти
        free_matrix(a);
        free_matrix(b);
        free_matrix(c);
    }

    return 0;
}

```

Файл 2lab_2task.c (2 задание)

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>
#include <locale.h>

//Объявления функций
char* get_type_name(int type);
void shell(int* items, int count);

```

```

void qs_iterative(int* items, int n);
int compare(const void* a, const void* b);
void generate_data(int* arr, int n, int type);
void test_sort(char* name, void (*sort_func)(int*, int), int* arr,
int n, int type);
void test_qsort(int* arr, int n, int type);

//Функция возвращает название типа данных для вывода
char* get_type_name(int type) {
    switch (type) {
        case 0: return "Случайные";
        case 1: return "Возрастающая";
        case 2: return "Убывающая";
        case 3: return "Смешанная";
        default: return "Неизвестно";
    }
}

//Реализация сортировки Шелла
void shell(int* items, int count) {
    int i, j, gap, k;
    int x, a[5] = { 9, 5, 3, 2, 1 }; //Последовательность
промежутков
    for (k = 0; k < 5; k++) {
        gap = a[k]; //Текущий промежуток
        for (i = gap; i < count; ++i) {
            x = items[i]; //Запоминаем текущий элемент
            //Сдвигаем элементы, пока не найдем правильную позицию
            for (j = i - gap; (x < items[j]) && (j >= 0); j = j -
gap)
                items[j + gap] = items[j];
            items[j + gap] = x; //Вставляем элемент на правильное
место
        }
    }
}

// Итеративная версия быстрой сортировки-без рекурсии
void qs_iterative(int* items, int n) {
    //Выделяем память для стека-границы подмассивов
    int* stack = (int*)malloc(n * 2 * sizeof(int));
    int top = -1; //Вершина стека

    //Помещаем в стек начальные границы всего массива
    stack[++top] = 0;
    stack[++top] = n - 1;

    //Пока стек не пуст
    while (top >= 0) {
        //Извлекаем границы текущего подмассива
        int right = stack[top--];
        int left = stack[top--];
        int i = left, j = right;
        //Выбираем опорный элемент (середина массива)
        int pivot = items[(left + right) / 2];

```

```

//Разделение массива относительно опорного элемента
do {
    while (items[i] < pivot) i++; //Ищем элемент больше
опорного слева
    while (pivot < items[j]) j--; //Ищем элемент меньше
опорного справа
    if (i <= j) {
        //Обмен элементов
        int temp = items[i];
        items[i] = items[j];
        items[j] = temp;
        i++; j--;
    }
} while (i <= j);

//Помещаем в стек границы левого подмассива (если он есть)
if (left < j) {
    stack[++top] = left;
    stack[++top] = j;
}
//Помещаем в стек границы правого подмассива (если он есть)
if (i < right) {
    stack[++top] = i;
    stack[++top] = right;
}
}
free(stack);
}

//Функция сравнения для стандартной qsort
int compare(const void* a, const void* b) {
    return (*(int*)a - *(int*)b); //Сравнение двух чисел
}

//Генерация тестовых данных разных типов
void generate_data(int* arr, int n, int type) {
    switch (type) {
        case 0: //Случайные числа
            for (int i = 0; i < n; i++) arr[i] = rand() % 10000;
            break;
        case 1: //Возрастающая последовательность
            for (int i = 0; i < n; i++) arr[i] = i;
            break;
        case 2: //Убывающая последовательность
            for (int i = 0; i < n; i++) arr[i] = n - i;
            break;
        case 3: //Смешанный тип: первая половина возрастает, вторая
убывает
            for (int i = 0; i < n / 2; i++) arr[i] = i;
            for (int i = n / 2; i < n; i++) arr[i] = n - i;
            break;
    }
}

//Тестирование алгоритма сортировки

```



```

void test_sort(char* name, void (*sort_func)(int*, int), int* arr,
int n, int type) {
    //Создаем временную копию массива для тестирования
    int* temp = (int*)malloc(n * sizeof(int));
    if (temp == NULL) return;
    memcpy(temp, arr, n * sizeof(int)); //Копируем исходные данные

    //Засекаем время
    clock_t start = clock();
    if (strcmp(name, "Быстрая") == 0) {
        qs_iterative(temp, n);
    }
    else {
        sort_func(temp, n);
    }
    clock_t end = clock();

    //Вычисляем время в секундах и выводим результат
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("%-10s | %-12s | %7d | %10.5f\n",
        name, get_type_name(type), n, time_taken);
    free(temp);
}

//Тестирование стандартной функции qsort
void test_qsort(int* arr, int n, int type) {
    //Создаем временную копию массива
    int* temp = (int*)malloc(n * sizeof(int));
    if (temp == NULL) return;
    memcpy(temp, arr, n * sizeof(int));

    //Измеряем время выполнения qsort
    clock_t start = clock();
    qsort(temp, n, sizeof(int), compare);
    clock_t end = clock();

    //Выводим результаты
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("%-10s | %-12s | %7d | %10.5f\n",
        "qsort", get_type_name(type), n, time_taken);

    free(temp);
}

int main() {
    setlocale(LC_ALL, "");
    srand(time(NULL));
    //Размеры массивов для тестирования
    int sizes[] = { 1000, 5000, 10000, 50000, 100000 };
    int num_sizes = sizeof(sizes) / sizeof(sizes[0]);
    printf("Алгоритм | Тип данных | Размер | Время(сек)\n");
    printf("-----\n");

    //Тестирование для каждого размера массива
    for (int size_idx = 0; size_idx < num_sizes; size_idx++) {
        int n = sizes[size_idx];

```

```

int* arr = (int*)malloc(n * sizeof(int));
if (arr == NULL) continue;

//Тестирование для каждого типа данных
for (int type = 0; type < 4; type++) {
    generate_data(arr, n, type);
    test_sort("Шелла", shell, arr, n, type);
    test_sort("Быстрая", NULL, arr, n, type);
    test_qsort(arr, n, type);
    printf("-----\n");
}
free(arr);
}
return 0;
}

```

Текст к программе

1. 2lab_12task.c (1 задание)

Программа измеряет время умножения квадратных матриц разных размеров. Алгоритм имеет кубическую сложность $O(n^3)$ — три вложенных цикла перемножают элементы матриц. Для матрицы размером n на n выполняется n^3 операций умножения и сложения.

Программа тестирует размеры от 100 на 100 до 4000 на 4000. Результаты показывают, что с увеличением n время растет крайне быстро: для $n=4000$ время выполнения составляет уже минуты или даже часы. Это подтверждает теоретическую оценку сложности и демонстрирует неэффективность простого алгоритма для больших матриц.

2. 2lab_2task.c (2 задание)

Данная программа тестирует и сравнивает три алгоритма сортировки: сортировку Шелла, быструю сортировку и стандартную функцию `qsort`. Программа проверяет их производительность на четырех типах данных: случайные числа, возрастающая последовательность, убывающая последовательность и смешанный тип.

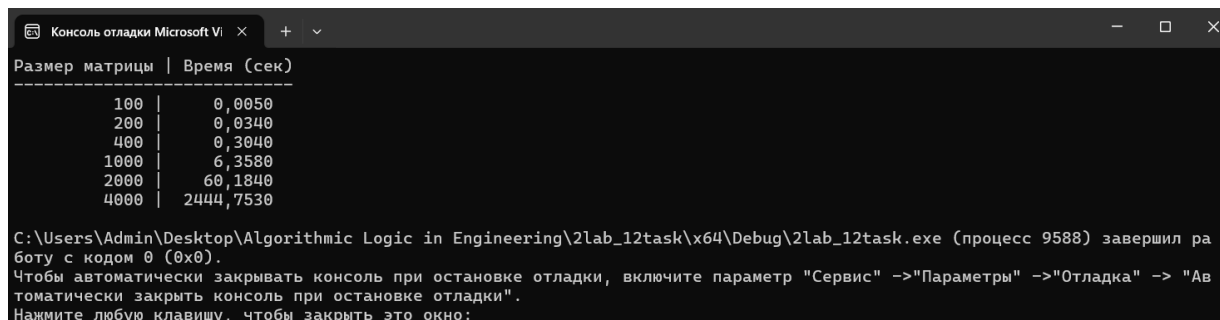
Для каждого размера массива (от 1000 до 100000 элементов) программа измеряет время выполнения всех трех алгоритмов. Быстрая сортировка реализована в итеративной версии для избежания переполнения стека при больших размерах данных.

Результаты выводятся в виде таблицы, где можно сравнить эффективность алгоритмов на разных типах данных и разных объемах. Это позволяет определить оптимальный алгоритм для конкретного типа данных и размера массива.

Результаты работы программы

Результаты работы программы показаны на рисунках 1-12.

I. 2lab_12task.c (1 задание)



```
Консоль отладки Microsoft Vi
Размер матрицы | Время (сек)
-----
100 | 0,0050
200 | 0,0340
400 | 0,3040
1000 | 6,3580
2000 | 60,1840
4000 | 2444,7530

C:\Users\Admin\Desktop\Algorithmic Logic in Engineering\2lab_12task\x64\Debug\2lab_12task.exe (процесс 9588) завершил ра
боту с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Ав
томатически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

Рисунок 2 – Корректная работа программы

II. lab_2task.c (2 задание)

Шелла	Случайные	1000	0,00000
Быстрая	Случайные	1000	0,00000
qsort	Случайные	1000	0,00000
Шелла	Возрастающая	1000	0,00000
Быстрая	Возрастающая	1000	0,00000
qsort	Возрастающая	1000	0,00000
Шелла	Убывающая	1000	0,00000
Быстрая	Убывающая	1000	0,00000
qsort	Убывающая	1000	0,00100
Шелла	Смешанная	1000	0,00000
Быстрая	Смешанная	1000	0,00100
qsort	Смешанная	1000	0,00000
Шелла	Случайные	5000	0,00200
Быстрая	Случайные	5000	0,00100
qsort	Случайные	5000	0,00100
Шелла	Возрастающая	5000	0,00000
Быстрая	Возрастающая	5000	0,00000
qsort	Возрастающая	5000	0,00000
Шелла	Убывающая	5000	0,00500
Быстрая	Убывающая	5000	0,00000
qsort	Убывающая	5000	0,00100
Шелла	Смешанная	5000	0,00300
Быстрая	Смешанная	5000	0,01600
qsort	Смешанная	5000	0,00100
Шелла	Случайные	10000	0,00900
Быстрая	Случайные	10000	0,00100
qsort	Случайные	10000	0,00200
Шелла	Возрастающая	10000	0,00100
Быстрая	Возрастающая	10000	0,00100
qsort	Возрастающая	10000	0,00100
Шелла	Убывающая	10000	0,01800
Быстрая	Убывающая	10000	0,00000
qsort	Убывающая	10000	0,00100
Шелла	Смешанная	10000	0,01200
Быстрая	Смешанная	10000	0,05800
qsort	Смешанная	10000	0,00400
Шелла	Случайные	50000	0,23100
Быстрая	Случайные	50000	0,00500
qsort	Случайные	50000	0,01000
Шелла	Возрастающая	50000	0,00100
Быстрая	Возрастающая	50000	0,00200
qsort	Возрастающая	50000	0,00500
Шелла	Убывающая	50000	0,50800
Быстрая	Убывающая	50000	0,00200
qsort	Убывающая	50000	0,00500
Шелла	Смешанная	50000	0,23500
Быстрая	Смешанная	50000	1,51800
qsort	Смешанная	50000	0,01800
Шелла	Случайные	100000	0,92800
Быстрая	Случайные	100000	0,01000
qsort	Случайные	100000	0,01900
Шелла	Возрастающая	100000	0,00200
Быстрая	Возрастающая	100000	0,00400
qsort	Возрастающая	100000	0,01000
Шелла	Убывающая	100000	1,85500
Быстрая	Убывающая	100000	0,00400
qsort	Убывающая	100000	0,01100
Шелла	Смешанная	100000	0,92700
Быстрая	Смешанная	100000	5,00900
qsort	Смешанная	100000	0,03700

Рисунок 3 – Корректная работа программы

Выводы

Оба кода подтверждают теоретическую оценку сложности алгоритмов на практике. Умножение матриц показывает кубический рост времени $O(n^3)$ — при увеличении размера матрицы в 2 раза время выполнения возрастает в 8 раз. Сравнение сортировок выявляет, что qsort — наиболее универсальный алгоритм, быстрая сортировка эффективна на случайных данных, но уязвима на специфичных наборах, а сортировка Шелла стабильна, но медленнее на больших массивах. Выбор алгоритма критически зависит от размера и типа данных.